LEARN BEING FORWARD

Programming — C Programming

Syntax, variables & data types

COMPLETE LEARNING GUIDE

What You'll Learn

C is the language most other imperative languages borrow their syntax and core semantics from, so getting its fundamentals exactly right pays off far beyond C itself — C++, C#, Java, and even JavaScript all inherit its statement structure, operator set, and much of its type-conversion behavior. Unlike higher-level languages, C gives you almost no safety net: the compiler won't stop you from reading an uninitialized variable, overflowing a signed integer, or truncating a float to an int, so understanding exactly what each type can hold and how values convert between types is not optional background knowledge — it's the difference between code that works and code that silently corrupts data. This sub-module covers the compilation pipeline, variable declarations and scope, the primitive type set and its platform-dependent sizes, type modifiers, implicit and explicit conversion, and the operator set, each grounded in a compilable example.

1. Program Structure and the Compilation Pipeline

A C program is built from functions, with exactly one entry point named main, and every file that uses a standard library function must #include the header that declares it. Unlike interpreted languages, C source goes through four distinct stages before it runs: the preprocessor expands #include and #define directives into plain C text, the compiler translates that text into assembly/object code, the linker resolves references between object files and library code into a single executable, and only then does the OS load and run it. Understanding these stages matters because errors surface at different points — a missing header produces a compiler warning about an undeclared function, while a missing library produces a linker error, and the two require completely different fixes.

- gcc -c main.c produces an object file (compile only); gcc main.o -o main links it into an executable.
- #include <stdio.h> pulls in declarations, not the actual printf implementation — that's resolved later, at link time.

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0; // 0 conventionally signals success to the OS
}

/* Compile and run:
   gcc hello.c -o hello
   ./hello
*/
```

2. Variables, Declarations, and Scope

A declaration tells the compiler a variable's name and type; a definition also reserves storage for it — in C these usually happen together (int x;), but the distinction matters once you work across multiple files with extern. Scope determines where a name is visible: variables declared inside a block ({ }) are block-scoped and only exist from their declaration to the closing brace, variables declared outside any function are file-scoped (visible to every function below them in that file), and each function parameter is scoped to that function body. C (since C99) allows declaring variables anywhere inside a block, not only at the very top, which makes it easy to keep a variable's declaration close to its first use.

- A variable declared inside an if block does not exist outside that block, even inside the same function.
- Always initialize local variables explicitly — unlike global variables (which default to 0), local variables hold indeterminate garbage until assigned.

```c
#include <stdio.h>

int global_counter = 0; // file scope, auto-initialized to 0

int main(void) {
    int local_total = 10;  // block scope: visible only inside main
    {
        int local_total = 99; // a NEW variable, shadows the outer one in this inner block
        printf("inner: %d\n", local_total); // 99
    }
    printf("outer: %d\n", local_total); // 10 — unaffected by the inner shadow
    return 0;
}
```

3. Primitive Data Types and Their Sizes

C's core primitive types are char (at least 8 bits, holds one byte), int (the natural integer size for the platform, typically 32 bits), float (single-precision IEEE 754, typically 32 bits), and double (double-precision, typically 64 bits). Critically, the C standard guarantees only MINIMUM sizes, not exact ones — int is guaranteed to be at least 16 bits but is 32 bits on virtually every modern desktop/server platform, and assuming a fixed size without checking is a classic portability bug. The sizeof operator reports the exact size (in bytes) of a type or variable on the current platform at compile time, which is the only reliable way to know a type's actual size rather than assuming it.

- Never hard-code assumed sizes like 'an int is always 4 bytes' in portable code — use sizeof or the fixed-width types in <stdint.h> (int32_t, uint64_t, etc.) when an exact width is required.

```c
C
#include <stdio.h>

int main(void) {
    printf("char:   %zu byte(s)\n", sizeof(char));
    printf("int:    %zu byte(s)\n", sizeof(int));
    printf("float:  %zu byte(s)\n", sizeof(float));
    printf("double: %zu byte(s)\n", sizeof(double));
    return 0;
}
/* Typical output on a 64-bit machine: char 1, int 4, float 4, double 8 */
```

4. Type Modifiers: signed, unsigned, short, long

signed and unsigned control whether a type's top bit represents sign or extends the magnitude — an unsigned int on a 32-bit platform ranges from 0 to 4,294,967,295 instead of int's roughly ±2.1 billion, trading negative numbers for double the positive range. short and long adjust the storage size relative to int (short int is often 16 bits, long int often 32 or 64 bits depending on platform). Unsigned arithmetic wraps around on overflow by definition (well-defined modulo 2^n behavior), which makes it tempting to use for 'this can never be negative' quantities like array sizes — but that same wraparound turns an accidental subtraction below zero into a silent, enormous positive number instead of a negative one, which is a frequent source of hard-to-find bugs.

- unsigned int x = 0; x--; does NOT trigger an error — x silently becomes 4294967295 on a 32-bit unsigned int.
- Signed integer overflow (unlike unsigned) is undefined behavior in C — the compiler is allowed to assume it never happens, which can produce surprising optimizations.

```c
#include <stdio.h>

int main(void) {
    unsigned int count = 0;
    count = count - 1; // wraps around instead of going negative
    printf("count = %u\n", count); // 4294967295 on a typical 32-bit unsigned int
    return 0;
}
```

5. Type Conversion: Implicit and Explicit (Casting)

C automatically performs implicit conversions in mixed-type expressions via the 'usual arithmetic conversions' — the operand with the narrower/lower-ranked type is promoted to match the wider one before the operation happens, so char and short operands are promoted to int, and mixing int with double promotes the int to double. The most common pitfall is integer division: 7 / 2 evaluates entirely in integer arithmetic and truncates to 3, discarding the fractional part, even if the result is immediately assigned to a double. Explicit casts, written as (type)expression, force a conversion at a specific point and are the correct fix — casting one operand to double before dividing forces the whole expression into floating-point arithmetic.

- 7 / 2 is 3 (integer division); (double)7 / 2 or 7 / 2.0 is 3.5 (floating-point division).
- Casting a larger floating type down to a narrower integer type (e.g., (int)3.99) truncates toward zero — it does not round.

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    printf("%d\n", a / b);              // 3 — integer division truncates
    printf("%f\n", (double)a / b);      // 3.500000 — one operand cast to double first
    printf("%d\n", (int)3.99);          // 3 — truncation, not rounding
    return 0;
}
```

6. Constants and the const / #define Distinction

A #define macro is a pure text-substitution performed by the preprocessor before compilation even begins — #define MAX 100 literally replaces every occurrence of MAX with 100 in the source text, so it has no type and the compiler never sees the name 'MAX' at all, only the number. A const-qualified variable (const int max = 100;) is a real, typed variable that the compiler enforces as read-only and that shows up in debuggers and error messages by name. Modern C style generally prefers const (or enum for related integer constants) over #define for simple value constants, reserving macros for cases that genuinely need text substitution or conditional compilation.

- #define SQUARE(x) x*x is a classic macro bug: SQUARE(1+2) expands to 1+2*1+2 = 5, not 9 — always parenthesize macro parameters and the whole expansion: #define SQUARE(x) ((x)*(x)).
- enum { RED, GREEN, BLUE } gives a readable, typed way to name a related group of integer constants (RED=0, GREEN=1, BLUE=2 by default).

```c
#include <stdio.h>

#define BAD_SQUARE(x) x*x
#define GOOD_SQUARE(x) ((x)*(x))

int main(void) {
    const int max_attempts = 3; // typed, debugger-visible, enforced read-only
    printf("%d\n", BAD_SQUARE(1+2));  // 5  — expands to 1+2*1+2
    printf("%d\n", GOOD_SQUARE(1+2)); // 9  — expands to ((1+2)*(1+2))
    printf("max_attempts = %d\n", max_attempts);
    return 0;
}
```

7. Operators and Expression Evaluation

C's operator set spans arithmetic (+ - * / %), relational (< <= > >= == !=), logical (&& || !), and bitwise (& | ^ ~ << >>) operators, each with a precedence level that determines evaluation order when operators are mixed without parentheses. Bitwise operators work on the binary representation of integers directly and are the standard way to pack multiple boolean flags into a single integer using OR to set a flag, AND to test it, and AND-with-complement to clear it — a pattern used constantly in embedded programming, graphics, and systems code. It's worth memorizing that & and | are bitwise (operate per-bit) while && and || are logical (operate on whole truth values with short-circuit evaluation) — confusing a single & for && is a common, hard-to-spot bug since both compile without error.

- Short-circuit evaluation: in (a != 0) && (10/a > 1), the division never executes if a is 0, because && stops evaluating once the left side is false.
- x & 1 tests whether x is odd (checks the lowest bit); x << 1 is equivalent to multiplying x by 2 for non-negative integers.

```c
#include <stdio.h>

#define FLAG_READ    (1 << 0) // 0001
#define FLAG_WRITE   (1 << 1) // 0010
#define FLAG_EXECUTE (1 << 2) // 0100

int main(void) {
    int permissions = FLAG_READ | FLAG_WRITE; // set READ and WRITE bits: 0011

    if (permissions & FLAG_WRITE) {
        printf("Write permission is set\n");
    }

    permissions &= ~FLAG_WRITE; // clear the WRITE bit: 0001
    printf("permissions now = %d\n", permissions); // 1
    return 0;
}
```

Common Pitfalls

- Assuming int is always exactly 4 bytes across every platform instead of checking with sizeof or using <stdint.h> fixed-width types.
- Using unsigned for a value that can legitimately go negative in an intermediate calculation, causing silent wraparound instead of a visible negative result.
- Expecting integer division (7/2) to produce a fractional result without casting at least one operand to a floating type.
- Writing unparenthesized macro parameters (#define SQUARE(x) x*x), which breaks silently when called with an expression argument.
- Reading a local variable before it has been explicitly assigned a value, relying on it 'probably' being zero the way global variables are.
- Confusing bitwise & / | with logical && / || — both compile without error but produce very different results.

Key Takeaways

- The compilation pipeline has four distinct stages (preprocess, compile, link, load) — knowing which stage an error comes from tells you what to fix.
- C only guarantees minimum type sizes, not exact ones; use sizeof or <stdint.h> types when exact width matters.
- Local variables are NOT auto-initialized — always assign a value before first use.
- Signed overflow is undefined behavior; unsigned overflow is well-defined wraparound — know which type you're working with before relying on overflow behavior.

- Prefer const or enum over #define for simple value constants; reserve macros for genuine text substitution.
- Bitwise operators (&, |, ^, ~, <<, >>) operate per-bit and are the standard tool for packing and testing flags in a single integer.

Learn More

cppreference — C Language Reference
https://en.cppreference.com/w/c/language

GeeksforGeeks — Data Types in C
https://www.geeksforgeeks.org/c/c-data-types/